Project Report: Handmade Crochet E-Commerce Website

1. Overview

This project is a fully functional, single-page website built for a real handmade crochet business. It was designed and developed independently, covering the full spectrum of a small e-commerce storefront: brand presentation, product browsing, order intake, payment routing, and customer engagement — all without a traditional backend server.

The site demonstrates practical, applied web development skills: responsive front-end design, third-party API integration (Formspree, Firebase, PayPal), form validation and UX fallback handling, and thoughtful business-logic implementation (pricing, delivery calculation, order routing).

2. Objectives

The site was built to solve concrete problems for a small handmade-goods business:

- Provide a professional, branded online storefront without the cost or complexity of a full e-commerce platform (e.g. Shopify)
- Allow customers to browse existing designs and place both **fixed-price** and **custom** orders through the same interface
- Automate order communication (confirmation emails, cancellation requests) without manual admin work
- Display genuine customer reviews to build trust, updating live without needing a page rebuild or database admin panel
- Keep hosting and running costs at zero by relying entirely on static hosting plus free-tier third-party services

3. Architecture & Design Decisions

3.1 Static-first approach

The entire site is a single self-contained HTML file with embedded CSS and JavaScript. This was a deliberate choice: it keeps hosting trivial (works on GitHub Pages, Netlify, or any basic web host), avoids server maintenance, and keeps load times fast since there's no framework overhead.

3.2 Serverless order handling

Rather than building a backend to receive and store orders, the site integrates with **Formspree**, a form-to-email service. Two separate Formspree endpoints are used — one for custom orders and product purchases, one for cancellations — keeping the two workflows cleanly separated on the business side.

A key design decision here was **graceful degradation**: if the Formspree request fails (network issue, service downtime), the JavaScript automatically falls back to opening the customer's email client with a pre-filled mailto: link containing all the order details. This ensures no order is ever silently lost, at the cost of the reference photo (which can't be attached via mailto: — the fallback path explicitly asks the customer to re-attach it by replying to the email).

3.3 Live reviews via Firebase Firestore

Customer reviews needed to feel "live" — appearing instantly without a page refresh or manual moderation step — while still preventing abuse (editing or deleting other people's reviews). This was solved with Firebase Firestore, configured with security rules that permit public read and create operations but explicitly deny update and delete:

```
match /reviews/{reviewId} {
  allow read: if true;
  allow create: if true;
  allow update, delete: if false;
}
```

When a review is submitted, it's written to Firestore and immediately prepended to the on-page review list via direct DOM manipulation, so the submitter sees their review appear without waiting on a database round-trip for the *read*. A duplicate copy of the review is also emailed to the shop owner via Formspree, so review activity doesn't require actively checking the Firebase console.

The integration is defensive: if FIREBASE_CONFIG is left as a placeholder or the Firebase SDK isn't loaded, the review-loading and review-display logic silently no-ops rather than throwing errors that would break the rest of the page.

3.4 Product pricing logic

Each product category (flowerpots, coasters, flower bouquets, keychains, earrings) has its own quantity-based pricing table, with delivery cost calculated and displayed alongside the subtotal in real time as the customer changes quantity — implemented via small updateShopPrice()-style handlers tied to each product's quantity selector. A free-delivery threshold (orders of £25+) is surfaced via a banner, encouraging larger orders.

3.5 Dual checkout paths

Customers can either:

1. **Buy Now** — routed directly to a pre-generated PayPal payment link specific to that product and quantity combination, or

2. **Add to custom order** — routed into the general order form, pre-filled with their product selection, for items needing a fixed price to be confirmed manually (the form displays a note explaining that a secure payment link will follow by email once details are confirmed)

This two-track approach lets simple, standardized items (like a fixed set of keychains) skip the back-and-forth of manual quoting, while genuinely custom items (specific colours, sizes, or one-off requests) still get a human review before payment.

4. Key Features Summary

Feature	Implementation
Responsive landing page & gallery	HTML/CSS, custom design tokens (Fraunces/Quicksand fonts, warm neutral palette)
Product shop with live pricing	Vanilla JS event handlers per product card
Direct checkout	Pre-generated PayPal payment links per product/quantity
Custom order intake	HTML form → Formspree → email, with mailto: fallback
Order cancellation	Separate Formspree endpoint and form
Live customer reviews	Firebase Firestore (public read/create, no update/delete) + Formspree copy to owner
Confirmation emails	Triggered client-side on successful order submission

5. Challenges & Solutions

- **No backend, but needed persistence** → solved with Firebase Firestore for reviews and Formspree for order/cancellation delivery, avoiding the need to run or pay for a server.
- **Reliability of third-party form service** → solved with an automatic mailto: fallback so a Formspree outage never means a lost order.
- **Preventing review tampering while keeping it open** → solved with Firestore security rules restricting the collection to append-only writes.
- **Balancing fixed-price simplicity with custom-order flexibility** → solved with the dual Buy Now / Add to Custom Order paths described above.

6. Possible Future Improvements

- Migrate from a single monolithic HTML file to a lightweight framework (e.g. Astro or plain component splitting) as the product catalog grows, for easier maintenance

- Add basic admin tooling (e.g. a Firebase-authenticated dashboard) so the shop owner can moderate reviews without needing direct Firestore console access
- Add inventory/stock tracking so sold-out custom slots or limited-run items are reflected automatically
- Add order status tracking for customers (e.g. a lookup by order ID) rather than relying purely on email updates

7. Conclusion

This project delivers a complete, low-cost, and maintenance-light e-commerce presence for a handmade goods business, built entirely with front-end technologies and free-tier third-party services. It demonstrates end-to-end thinking about a real business's needs — not just visual design, but order handling reliability, payment routing, and customer trust features like live reviews — while keeping the technical footprint deliberately simple.

Tools used: HTML5, CSS3, JavaScript (ES6+), Formspree, Firebase Firestore, PayPal payment links, Google Fonts